Parcel damage inspection

Jimena Navarro del Águila — Assignment 2

Decision and scope

I selected a visible parcel damage inspection and a feedback loop. The prototype is designed to help a warehouse operator decide whether a parcel can pass, should be held for inspection, or needs a person to check it. The positive class is box_damaged; box_opened remains a separate detection class and is not silently relabelled as damage. The policy concerns visible external damage, not proof of internal product condition or transport safety.

Dataset provenance

Public Roboflow Universe workspace recog-ayum6, project boxes_status, version 2, exported as yolov8. Source: https://universe.roboflow.com/recog-ayum6/boxes_status/dataset/2 . Licence: CC BY 4.0; attribution to Recog and the source project.

The supplied train/validation/test splits and source labels are retained unchanged. No private fork is used.

The exported splits contain 389 training, 104 validation and 52 test images. Instance counts, relative object sizes and duplicate checks are in artifacts/dataset-health.json and MEASURED_RESULTS.md. The exact-byte duplicate audit found one cross-split match: [['train', 'val', 'istockphoto-1215484506-612x612_jpg.rf.8ad8d2653228b0fe59d9218894bd52af.jpg']]. The public splits were preserved as required; the overlap is disclosed rather than removed. Thus, exact-byte checks do not detect all resized or near-duplicate source images.

The sample unit for the operational cost calculation is an inspection image, not necessarily one unique parcel but multi-box images make that distinction material.

Training and measurement record

YOLO11n is fine-tuned locally on an Apple M4 MacBook Air with 16 GB unified memory, using PyTorch MPS. Initial settings: 416 pixels, batch 8, seed 42, maximum 30 epochs and patience 8. The first run stopped early after 27 epochs, then encountered an apostrophe-related checkpoint loading failure. The saved checkpoint was recovered without training on test data and the pipeline stages checkpoint copies under an apostrophe-free temporary path to prevent this issue.

A candidate model was trained at higher image resolution, then compared with the original using validation cost and recall. And it’s important to mention that this is a validation experiment, not evidence of an autonomous discovery of a causal failure mechanism. The test images are kept out of both training and model selection. Final measured tables are generated separately in MEASURED_RESULTS.md and artifacts/.

Business assumptions

The starting scenario was authorised for investigation: EUR 40 per missed visible damage, EUR 2 per unnecessary hold, EUR 2 per correct preventive hold, 10 percent prevalence and 2,000 inspection images per day (warehouse assumptions, not measured loss or prevalence estimates). BUSINESS_INPUTS.md records the benchmark sources, assumptions and sensitivity ranges.

We assume a person takes 30 seconds to check one image. At a labour cost of €26.18 per hour, that works out to about €0.22 per check: €26.18 × 30 ÷ 3,600. This covers the reviewer's time, before any additional handling costs. The hourly figure is a general Spanish labour-cost benchmark, so it does not tell us what a particular warehouse actually pays.

We also assume the reviewer correctly identifies 99% of damaged images and correctly clears 99% of undamaged images. These figures are assumptions for comparing scenarios; we have not measured real reviewers' speed or accuracy.

If the team can spend two hours a day reviewing images, it can check 240 images at 30 seconds each. For staffing calculations, we count one person as providing six productive hours per day, so two review hours equal about one-third of a person's daily capacity. Six hours is a planning assumption, not a claim about the length of their shift.

Measured evidence

Real per-class detection results, same-test baselines, validation-selected thresholds, held-out expected costs, review volume and the candidate gate outcome are in the MEASURED_RESULTS.md file. Confidence scores are not calibrated damage probabilities and scenario-adjusted costs depend on the assumed production base rate and human performance.

Additional measured checks

The validation-selected policy uses thresholds 0.12 and 0.28. Under the specified scenario, held-out review demand is approximately 435.7 images/day, exceeding the assumed capacity of 240/day. Test damage-class AP50 is approximately 0.183 and summary recall approximately 0.115 at the detector's maximum-F1 operating point; which differs from the low-threshold image-level decision policy.

Analysis

What does the dataset say?

The dataset is useful for testing the idea, but it is small for deciding whether it would work in a real warehouse. There are 1,170 intact-box instances in training, compared with 248 damaged and 58 opened boxes, so the model has much more practice recognising the majority class. The test set has only 27 damaged and eight opened instances, which makes the class results sensitive to a few mistakes. There is also one identical image shared between training and validation, so the validation comparison is not fully independent. I kept the supplied splits unchanged and reported the overlap. Before using the system, I would need a separate evaluation using images from the actual inspection camera.

Does the model detect the damage well enough?

The overall AP50 is 0.523, but that average hides the main problem: intact boxes have AP50 of 0.751, while damaged boxes have only 0.183. Damage recall at the detector's maximum-F1 point is about 11.5%. That is too weak to rely on the detector alone for deciding that a parcel is safe to pass. The image-level policy uses lower thresholds and human review, therefore its scenario-adjusted sensitivity is a different measure; it does not mean that the detector accurately locates every damaged box. Both baselines score zero on these target labels because the COCO vocabulary has no matching parcel-status classes. Beating them shows that fine-tuning learned something, but does not establish readiness for use.

Why do the mistakes have different costs?

Missing visible damage can mean sending a parcel that later needs a refund, replacement or another delivery, whereas an unnecessary hold mainly uses time and slows the process down. That is why the starting scenario assigns EUR 40 to missed damage and EUR 2 to an unnecessary hold (assumptions to test, not actual warehouse accounts). A correct hold also costs EUR 2 because checking or handling a genuinely damaged parcel still takes resources. The EUR 26.18 hourly labour benchmark gives about EUR 0.218 for a 30-second check, but it is not a measured warehouse inspection cost.

I would replace these inputs with actual claims, handling and inspection records before making a business decision, and measure human accuracy instead of assuming 99 percent.

Which thresholds would I use?

The validation search selected 0.12 and 0.28: below 0.12 gives a pass recommendation, the middle range goes to a person, and 0.28 or above means hold for inspection. Images without a recognised box also go to a person; I would keep those thresholds frozen for reporting the test results rather than change them after seeing the test. Under the stated assumptions, the test cost is EUR 804.93 per 1,000 images, compared with EUR 1,100 for the model alone and EUR 4,000 for passing everything. Checking everything manually is cheaper at EUR 474.17 in this scenario, but needs much more staff time. Therefore, the model is not the cheapest option under every comparison.

Can the team handle the review workload?

The selected policy fits the two-hour review limit on validation, but not on the test set: it sends about 436 images a day to review, taking 3.63 hours rather than the two hours available. That is about 0.61 of a six-hour productive working day, while reviewing everything takes 16.67 hours, or 2.78 staffing equivalents. I would not present the current policy as workable with the original capacity; more reviewer time or a better model would need to be evaluated on fresh data. Changing the false-negative cost changes the cost comparison, while changing prevalence also changes the expected mix of errors and review workload; the saved sensitivity results keep the thresholds fixed so that these effects can be inspected without retuning on the test set.

Does the feedback loop improve the system?

The candidate used higher-resolution images and ten extra training epochs. On validation, its best model-only expected cost fell from EUR 856.88 to EUR 785 per 1,000 images, while image-level sensitivity rose from about 92.2 percent to 100 percent at the selected threshold. It passed the specified comparison and became the preferred candidate, but this does not prove it will recognise every new kind of damage. The loop records mistakes and tests a possible improvement; it cannot know the real reason for every error from an image alone. New corrections need verified labels and a separate evaluation, and a person still has to approve any replacement.

Would I use it now?

I would keep this as a prototype and test it alongside human inspection, rather than let it release parcels independently. The idea is useful, especially the feedback loop, but the damage-class result, small test sample and excessive review workload are reasons to continue testing. The next step would be to evaluate real camera images, record missed damage and unnecessary holds, then compare a new candidate on an independent set. Equally, a model replacement should depend on lower cost, acceptable damage recall and a workload the team can actually handle, not just a better overall score.

Feedback and replacement controls

Human-verified feedback is stored locally with the original image, correction, prediction, model version and thresholds; it is kept separate from the public coursework splits. A replacement requires verified labels, independent evaluation, acceptable cost and recall, capacity checks and human approval. Overall, the software records promotion evidence and rollback information; it does not automatically replace a production model.

Reproduction

Run python main.py train, then python main.py app in the configured Python environment, or use the documented uv commands after installing dependencies. The app loads saved weights and artifacts and never trains at startup. Save the private Roboflow key in .env; never include it in a ZIP or version control.

Assistance disclosure

I used AI to help write and debug the code, check calculations, summarise measurements, format the report and write interpretations from the results. I chose the parcel-damage problem and the feedback-loop direction, and reviewed the initial scenario assumptions.

Measured results

Split	Class	Instances	Median box area
train	box	1170	20.2%
train	box_damaged	248	16.7%
train	box_opened	58	22.6%
val	box	249	23.3%
val	box_damaged	85	16.1%
val	box_opened	15	9.1%
test	box	168	19.3%
test	box_damaged	27	13.5%
test	box_opened	8	45.8%

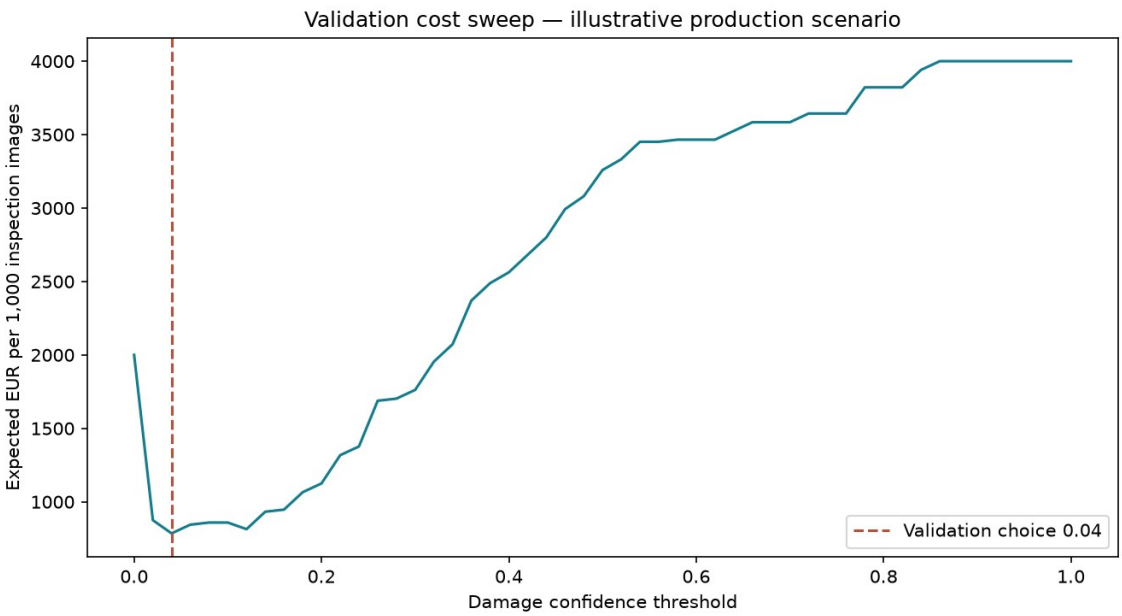
Per class detection

Class	Precision	Recall	AP50	mAP50–95
box	0.775	0.655	0.751	0.582
box_damaged	0.307	0.115	0.183	0.122
box_opened	0.586	0.625	0.635	0.574

Precision and recall use the detector maximum-F1 summary point. The separate image-level review policy uses its validation-selected thresholds.

Cost and staffing comparison

Policy	€/1,000 images	Reviews/day	Hours/day
model with review	804.93	435.7	3.63
model alone	1100.00	0.0	0.00
do nothing pass all	4000.00	0.0	0.00
human check everything	474.17	2000.0	16.67



Costs and staffing depend on the stated scenario assumptions. Full machine-readable results, confusion matrices and sensitivity checks are included in artifacts/.